# OpenMiniCrew

## Personal AI Assistant Framework

Claude + Gemini | Telegram Bot | Production Ready | Multi-user Ready

**Claude API** | **Gemini API** | **Telegram Bot** | **SQLite** | **APScheduler** | **Gmail/IMAP**

github.com/kaebmoo/openminicrew

# Background

| Problems | Solutions with OpenMiniCrew |
| --- | --- |
| OpenClaw has high risk -- too much capability without verification | Build a small, personal framework from scratch |
| Existing AI Agent tools are overly complex | Command via Telegram -- simple, immediate |
| Need tools with 100% personal control | Add tools with a single file, no core changes |
| Must be expandable without refactoring | Multi-user ready architecture from day one |

# What is OpenMiniCrew?

Personal AI assistant controlled via Telegram

### 2 Ways to Command

/command direct
and free-text chat

### 2 LLM Providers

Claude + Gemini
Switch anytime

### Tool System

Add with 1 file
Auto-discover registry

- Chat Memory -- remembers conversation context
- Rate Limiting + Retry Logic
- Cron Scheduler -- automatic scheduled tasks
- Gmail + IMAP Integration
- Webhook Production-Ready
- Multi-user Ready (expandable)

# Architecture

Deep Dive

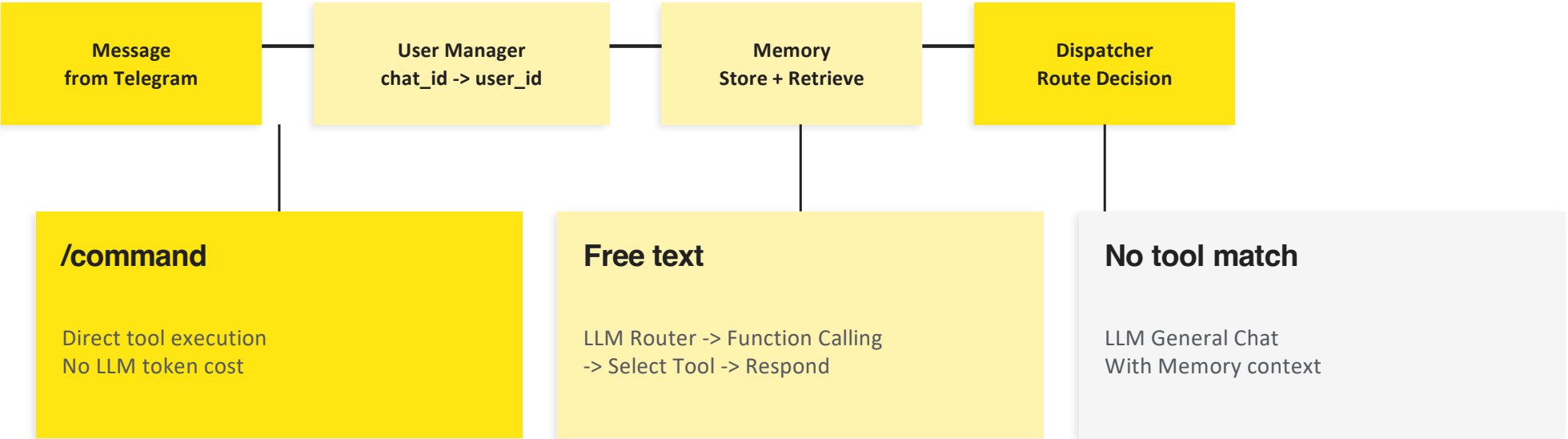
# Architecture Overview

| | |
|---|---|
| **INTERFACE LAYER** | Telegram Polling / Webhook + APScheduler |
| **USER MANAGER** | chat_id -> user_id \| authorization \| preferences |
| **DISPATCHER** | /command -> direct tool \| free text -> LLM -> tool \| fallback: LLM chat |
| **CORE LAYER** | LLM Router \| Tool Registry \| Memory Manager |
| **SHARED CORE** | config - db - memory - security - user_manager - logger |

tools/ email - traffic - places - news - work_email

providers/ Claude - Gemini - registry

# Dispatcher Flow




**Every case: Save Memory + tool_logs immediately for complete audit trail**

# Tool System

Add easily, no core changes needed

```
tools/my_tool.py

class MyTool(BaseTool):
    name = "my_tool"
    commands = ["/mytool"]

    async def execute(self,
            user_id, args):
        return "result"

    def get_tool_spec(self):
        return {
          "name": "my_tool",
          "description":"..."
```

## Auto-Discover Flow

1. Create file in tools/
2. registry.py scans all files
3. inspect finds BaseTool classes
4. Create instance + register
5. Ready for /command and LLM calling

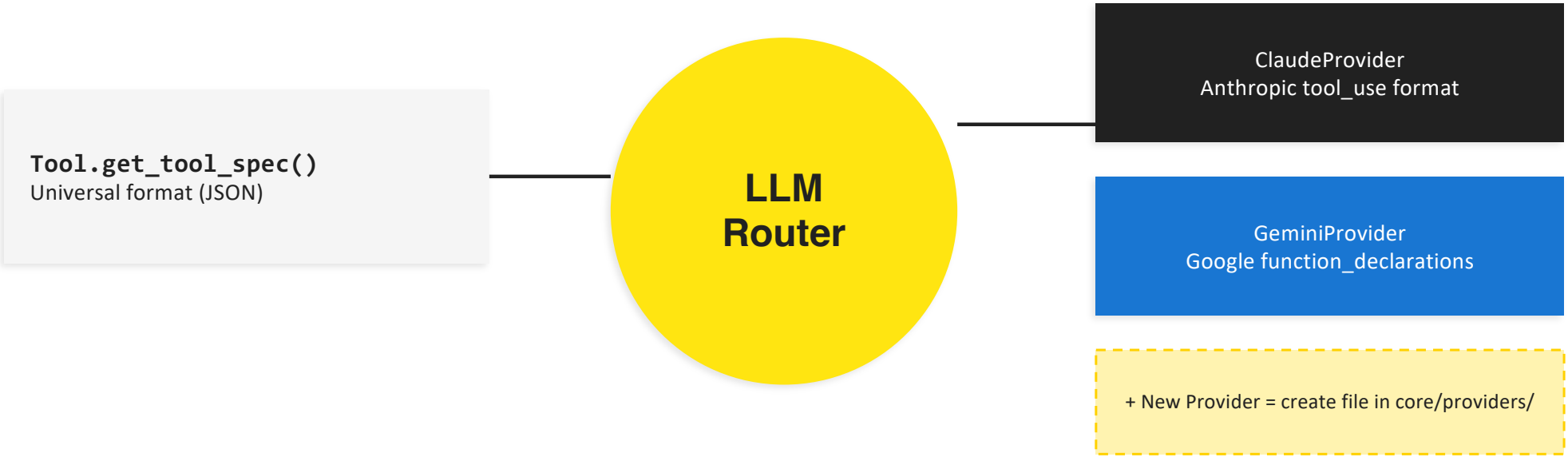
# Available Tools

| | | |
|---|---|---|
| `/email` | **email_summary** | Gmail summary with time filter, search, force rescan |
| `/wm` | **work_email** | Corporate email via IMAP + PDF/DOCX/XLSX attachment summary |
| `/traffic` | **traffic** | Route + real-time traffic via Google Maps (4 modes) |
| `/places` | **places** | Nearby places search via Google Places API (New) |
| `/news` | **news_summary** | Latest news from Google News RSS + LLM summary |

**+ Add new tools easily -- see Tool System slide for how**

# Multi-LLM Router

Choose provider freely, not locked to any one




- Write tool spec once, works with all providers
- Auto-fallback: if Claude fails -> try Gemini
- User switches LLM via /model claude or /model gemini
- Cost control: tier='cheap' uses Haiku/Flash | tier='mid' uses Sonnet/Pro

# Production-Ready Features

## Webhook Mode

- FastAPI responds 200 immediately
- BackgroundTask + secret_token verify
- /health endpoint

## Rate Limiting

- Token bucket rate limiter
- 30 msg/s Telegram limit
- Queue burst protection

## Retry Logic

- Auto-retry LLM API calls
- Exponential backoff
- Error notification to user

## Scheduler

- Single-process (APScheduler)
- SQLite jobstore
- Morning briefing 07:00 auto

## Memory + Cleanup

- N latest messages per user
- Retention policy 30 days
- Auto-prune daily job

## Structured Logging

- tool_logs per user
- token_used tracking
- Error details for debug

# Add New Tool -- 5 Steps

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| **Create file** | **Write class** | **Add dependencies** | **Add config** | **Restart bot** |
| touch tools/my_tool.py | inherit BaseTool<br>set name, description, commands | (optional) pip install library<br>add to requirements.txt | (optional) add to .env<br>and core/config.py | python main.py<br>Log: Registered tool: my_tool |

**Ready to use in 2 ways:**
/mytool -- direct tool call, no LLM token cost | Free text -- LLM function calling selects tool automatically

# Security Checklist

### User Authorization
chat_id -> user_id -> is_active + role-based access

### Webhook Secret Token
X-Telegram-Bot-Api-Secret-Token verification

### API Keys in .env
No hardcode -- .gitignore prevents commit

### HTTPS Only (Webhook)
Telegram enforces HTTPS, prevents MITM

### Gmail OAuth
Readonly scope + auto-refresh token

### No Shell Exec
Tools do only what is coded, no subprocess or eval()

### SQLite WAL Mode
Prevents database corruption from concurrent writes

### Rate Limiting
Prevents Telegram API rate limit + user spam

# Getting Started -- 4 Steps

## 1 Install

- git clone / copy project
- pip install -r requirements.txt
- cp .env.example .env

## 2 Telegram Bot

- Talk to @BotFather -> /newbot
- Copy Bot Token -> TELEGRAM_BOT_TOKEN
- Get Chat ID via @userinfobot

## 3 LLM API Keys

- console.anthropic.com -> ANTHROPIC_API_KEY
- aistudio.google.com -> GEMINI_API_KEY
- Set DEFAULT_LLM in .env

## 4 Run

- python main.py
- (First time: browser opens for Gmail auth)
- BOT_MODE=webhook for production

```
Startup: DB -> User init -> Gmail auth -> Tool discovery -> Scheduler -> Bot (polling/webhook)
```

# Multi-user Ready

Expandable without refactoring

| Today (Single User) | Future (Multi-user) |
| --- | --- |
| OWNER_TELEGRAM_CHAT_ID in .env | /start -> pending -> owner approve -> active |
| Gmail token of owner | OAuth callback for Gmail per-user |
| LLM API keys single set | /authorize_gmail via Telegram link |
| /adduser <chat_id> to add users | Admin: /users, /usage, /disable |

**No changes needed when expanding users:** tools - dispatcher - LLM router - memory - DB schema - scheduler - security

# Roadmap -- Tools That Can Be Added

| Command | Tool | API | Difficulty |
|---|---|---|---|
| `/weather` | Weather | Open-Meteo (free) | Easy |
| `/translate` | Translation | LLM direct (no extra API) | Easy |
| `/note` | Notes | SQLite (already available) | Easy |
| `/fx` | Exchange Rate | exchangerate-api (free) | Easy |
| `/search` | Web Search | Google Custom Search / SerpAPI | Medium |
| `/yt` | YouTube Summary | YouTube Transcript API + LLM | Medium |
| `/track` | Package Tracking | Thailand Post API / Kerry | Medium |
| `/cal` | Google Calendar | Google Calendar API | Hard |

# OpenMiniCrew

- Command AI via Telegram -- both /command and free text
- Supports Claude + Gemini, switch anytime, not locked to any provider
- Add new tool = create 1 file, no core changes
- Production-ready from day one -- webhook, rate limit, retry, health check
- Multi-user ready -- expandable without refactoring

**github.com/kaebmoo/openminicrew**

**Questions?**

สนับสนุนค่ากาแฟ ค่าเบียร์ได้ที่นี่

buymeacoffee.com/ogonan

buymeacoffee.com/ogonan